CS612 Lecture 06 — In-Class Exercises

Dynamic Programming I — State-Space Design · Sat 31 Oct 2026 · ungraded

For every DP you write, give: (1) the state in words, (2) the recurrence + base cases, (3) why it is correct (case analysis on the last decision + cut-and-paste), (4) the evaluation order, (5) the complexity as #states × transition cost.

A — Define the state (no code)

1. **Word break.** Can $s[1..n]$ be split into dictionary words? State, recurrence, cost (dictionary lookup cost ℓ; words of length at most L).
2. **Grid with few turns.** Count monotone (right/down) paths from $(1,1)$ to (n,n) that change direction at most k times. What must the state remember? How many states?
3. **Deadline tasks.** Tasks $1..n$ in order; task i takes t_i or is skipped at penalty p_i; total time $\leq T$; minimise the penalty. State? Is the algorithm polynomial?

Apply the **two-histories test** to each of your states.

B — Prove and refute

1. Find dimensions for 3 matrices for which "multiply the cheapest adjacent pair first" is not optimal.
2. Prove the cut-and-paste step for matrix-chain multiplication.
3. Compute the edit distance "kitten" → "sitting" and give an optimal alignment.

C1 — Design

1. **Longest common substring** (contiguous). Why does the LCS state not work unchanged? Give the right state and complexity.
2. **Cutting a stick** of length ℓ at positions $c_1 < \cdots < c_k$; each cut costs the length of the piece being cut. Minimise the total cost.
3. **Segmented least squares.** Cost = the sum of squared errors of the segments + C per segment. State and recurrence; how do you get each segment's error in $O(1)$?

C2 — Critique

1. **Circular robber:** $R[i] = \max(R[i-1], R[i-2] + v_i)$ on a circle of houses. What is wrong? Fix it without adding a dimension.
2. **LIS:** $L[i] = L[i-1] + 1$ if $a_i > a_{i-1}$, else $L[i-1]$. Counterexample and correct state.
3. **Grid path with a budget:** $V[i][j]$ = the best value at (i,j) among paths within budget. What does the two-histories test say?